Midterm — "Landed" · what to say (6 minutes)

Short sentences on purpose. Say them out loud once or twice and they will sound natural. The same text is in the speaker notes of the PowerPoint: open **Presenter View** (Option + Cmd + P).

Total: 6 minutes. Slide times add up to 350 s, so you have 10 s of slack.

1 · Landed

15 s

I'm Candela. My final project is called **Landed**. It helps international students who just arrived in Taiwan. One checklist for your first thirty days.

2 · The problem

40 s

Arriving here is hard, for three reasons.

First, the information is everywhere. Immigration, the university, the bank. Nobody has the full list.

Second, it gets old fast. You find a post from 2023, in Chinese, and the office has moved.

Third, there is a legal deadline. The ARC must be filed in fifteen days.

Right now students use three things. A PDF from the office. An old Facebook post. Or a friend who did it last year. None of them knows your dates.

3 · What it is

35 s

Landed turns that into one checklist. You answer five questions. You get only the tasks for your situation. Each task has a real date, counted from the day you land. The international office keeps the content updated. And every task shows when it was last checked.

The information already exists. What does not exist is the order, the dates and the trust.

4 · Users

45 s

Three users.

The **student** arrives with a visa and no plan. They need to know what to do, and when.

The **office** answers the same questions every semester. They need to publish once, in two languages.

And **senior students** know the tricks. So they can add tips — but someone approves them first.

5 · Requirements

30 s

Students answer the questions and tick off tasks. Staff write the tasks in English and Chinese. Senior students suggest tips.

The system does three things. It calculates your dates. It locks the tasks you cannot do yet. And it saves every change.

One more rule: it must work well on a phone. You read this standing in a queue.

6 · MVP

35 s

The must-have part is the loop. Questions, checklist, task, tick, next. Plus the back office.

Reminders and tips can wait. I also cut the document wallet. It would store passport scans. That needs a privacy review first.

7 · Structure

30 s

Two parts, one database. The student app has five screens. The back office is for the office staff. That is the part that keeps everything true.

8 · Flows

35 s

The student flow is a loop. Answer once. Then open, do, tick, next.

The staff flow keeps it alive. A rule changes, they edit both languages, they publish. And the verified date starts again.

9 · Screens

35 s

Two screens.

The checklist tells you what to do now. Due this week, already done, and locked. The bank account is grey, because you need the ARC first. That saves you a wasted trip.

The task page tells you what to bring, before you go.

10 · Design decisions

45 s · · the important one, go slow

Three decisions I want to explain.

First: five questions, not forty tasks. Nobody needs all of them.

Second: dropdowns, not free text. The rules can only match clean values.

Third: one record with both languages. You cannot publish with one side empty. If you use two pages, the Chinese one gets old.

11 · Data

35 s

Five tables. Tasks have both languages, and the rule that decides who sees them. Deadlines are saved as offsets, not dates. "Fifteen days after entry" becomes your real date. Progress is private to each student. Tips wait for approval. And the log saves every change.

12 · System

30 s

The app runs on the phone. The database is Supabase. The rules live in the database, not in the screen. Published tasks are public. Your progress is only yours. Only staff can edit. And this is the same stack I use in class. So the technical risk is already solved.

13 · Next

30 s

My next step is not code. First I talk to five students who arrived this semester, and to the office. Is the list right? Is the order right? Then I build the loop, then the back office. And I test it in February, with students who are really arriving.

My question for you: who should own the content? The office, or the students?

If they ask

Isn't this the university website?

The website has pages. This has a sequence. Personal, with dates, and you can see if it is still true.

Can't ChatGPT tell me all this?

It can, and it will sound right. But it does not know today's fee, it does not know your arrival date, and nobody is responsible if it is wrong. My product shows who verified each step, and when.

How do you keep it correct?

The office owns the content. Every task shows the last date it was checked. And students can report a step that is wrong.

Why only Fu Jen?

The rules change by university and by city office. One campus first, so I can verify everything. The database already allows more later.

What if a student answers wrong?

Only their own checklist changes. They can answer again and it regenerates.

Why not a mobile app?

A website works on day one. No download, any phone, and I can update it the same day.

What is the biggest risk?

The content, not the code. Forty tasks, in two languages, verified. That is why I talk to the office before I build.